

Sistemas de Gestión de Seguridad

Grado en Ingeniería Informática de Gestión y sistemas
de información

Sistema Web: Ataque Sistema Web

Proyecto víctima:

<https://github.com/pollitoDestructor/sgssi-allianz>

2025-2026

Fecha:

Bilbao, a 14 de noviembre de 2025

Grupo:

Podcast

Autores:

Fernández Pajares, María

Fernández Molano, Iker

Galván Carrillo, Nahia

Molinero Medel, Lucía

Rivera Urretxi, Aitzol

Tapias Monasterio, Paula

ÍNDICE

1. ACLARACIÓN INICIAL.....	4
2. PASOS PARA REALIZAR EL ATAQUE.....	4
2.1. CAMINO 1 (INYECCIÓN SQL).....	5
2.2. CAMINO 2 (MANIPULACIÓN DE PARÁMETROS EN URL).....	7
3. CONCLUSIÓN DEL ATAQUE.....	9

ÍNDICE DE FIGURAS

FIGURA 1.....4

FIGURA 2.....5

FIGURA 3.....5

FIGURA 4.....6

FIGURA 5.....7

FIGURA 6.....7

FIGURA 7.....8

1. ACLARACIÓN INICIAL

La instalación de los programas y paquetes que se van a utilizar durante el ataque está explicada en el archivo README.md del repositorio.

https://github.com/ikerfernandezmolano/PodcastSGSSI/tree/entrega_4

2. PASOS PARA REALIZAR EL ATAQUE

Inicialmente, accedimos al formulario de registro y completamos los campos habituales para crear una cuenta. Intentamos registrar un usuario utilizando un DNI, que por fortuna ya existía y la aplicación respondió mostrando un mensaje de error en la parte superior de la interfaz indicando la existencia del DNI. Se había detectado una inyección SQL basada en errores, ya que la aplicación mostró explícitamente que el DNI, que era clave primaria en la entidad, estaba duplicado, revelando información interna de la base de datos, como se muestra en la Figura 1.

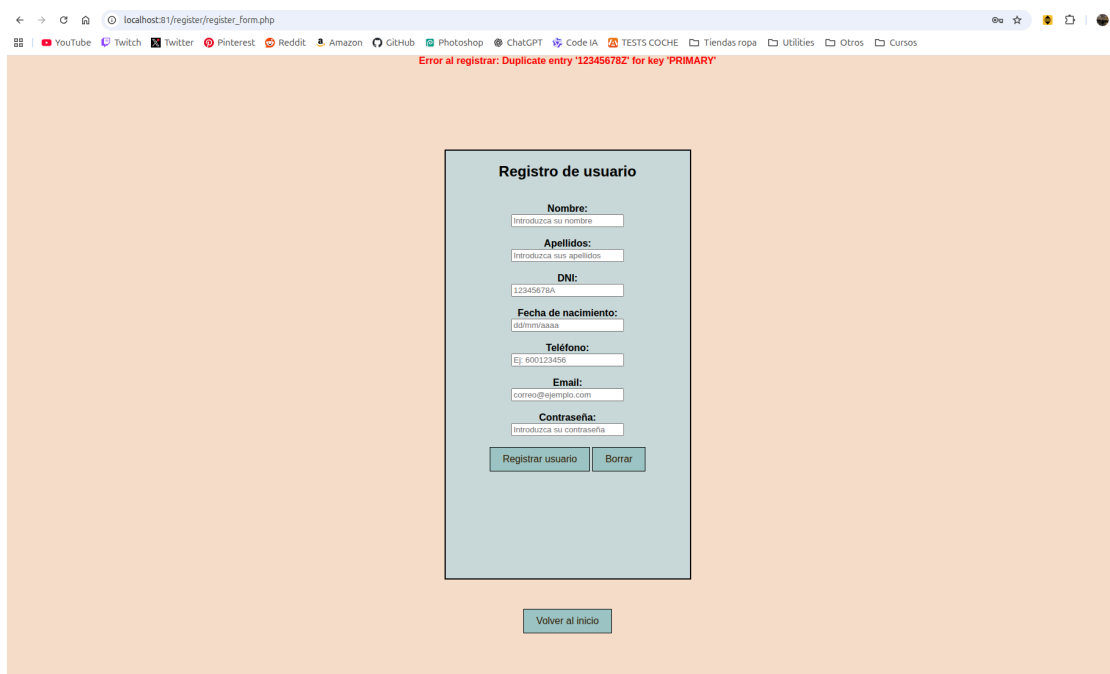


Figura 1.: Inyección SQL basada en error por DNI duplicado.

A continuación, se explicarán dos posibles caminos que hemos encontrado para obtener el usuario y contraseña (hasheada) de un usuario. El primer camino, está basado en inyección SQL. El segundo, en cambio, en la manipulación de los parámetros en las URL. Sin embargo, ambos llevan al mismo resultado.

2.1. CAMINO 1 (INYECCIÓN SQL)

Gracias a ZAP, que indicaba que había una posible inyección SQL (Figura 2), supimos de la existencia de la vulnerabilidad. A partir de esa información, utilizamos SQLMap para profundizar en ella, lo que nos permitió identificar exactamente el tipo de inyección presente y el parámetro concreto en el que se estaba produciendo.

El comando utilizado fue:

```
$ sqlmap -u "http://localhost:81/login/?" --crawl=3 --forms --batch -v 2
```

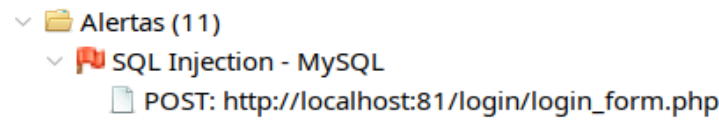


Figura 2.: Resultado de ZAP.

Por un lado, la aplicación web que analizamos presenta una vulnerabilidad de inyección SQL en el parámetro nombre, algo que pudimos confirmar gracias a varias técnicas que sqlmap fue probando automáticamente. Durante el análisis aparecieron distintos tipos de inyección —boolean-based blind, error-based, time-based blind y UNION-based— lo que dejó claro que la consulta SQL podía ser usada a nuestro favor para desvelar información, etc.

En la fase de pruebas, sqlmap detectó que el comportamiento del servidor cambiaba cuando se enviaban condiciones lógicas manipuladas, que era posible provocar errores intencionados en MySQL que dejaban escapar información y que incluso se podían introducir pausas forzadas usando funciones como SLEEP. Además, el uso de UNION SELECT permitió añadir consultas propias y obtener datos directamente de la base de datos. Todas estas señales confirmaron de forma definitiva la vulnerabilidad y nos permitieron extraer información sensible.

Por otro lado, detectamos que al intentar iniciar sesión usando un nombre de usuario válido pero con una contraseña incorrecta, la interfaz mostró una sección que exponía información adicional en forma de tabla. En esa tabla se visualizaba el identificador del usuario, el nombre y la contraseña encriptada mediante hash (Figura 3).

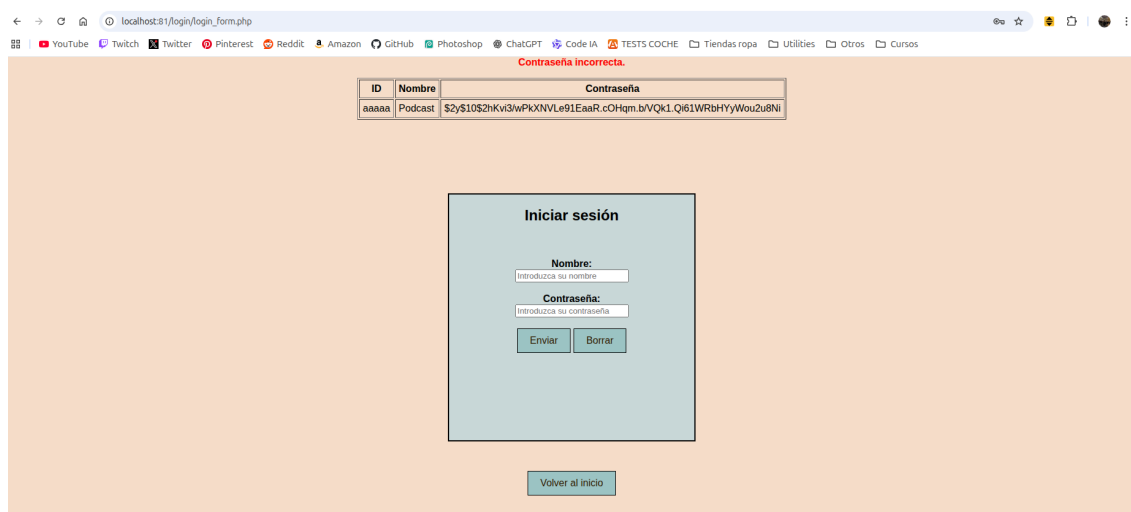


Figura 3. Tabla con nombre y contraseña hasheada del usuario admin.

Por lo que, combinando la inyección SQL en el parámetro nombre y la exposición de datos sensibles que presentaba el sitio web. Mediante la siguiente carga maliciosa introducida en el campo nombre:

```
' UNION ALL SELECT nombre FROM usuarios LIMIT 1 #&
```

Obtuvimos una tabla, mostrando el nombre y contraseña hasheada, en nuestro caso, la información correspondía al usuario admin. Revelando información de gran sensibilidad (Figura 4).

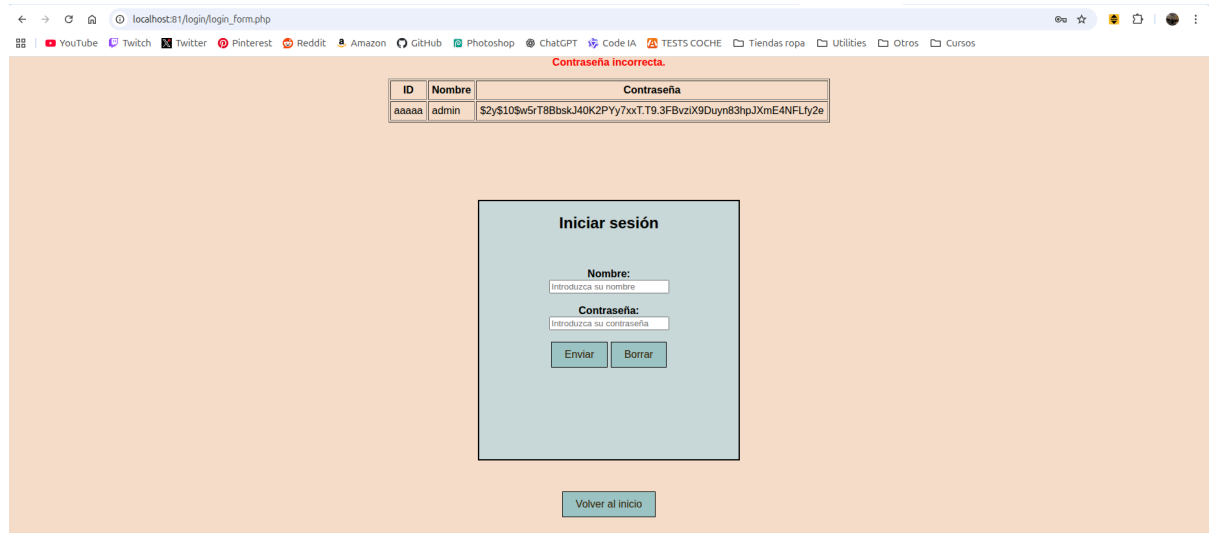
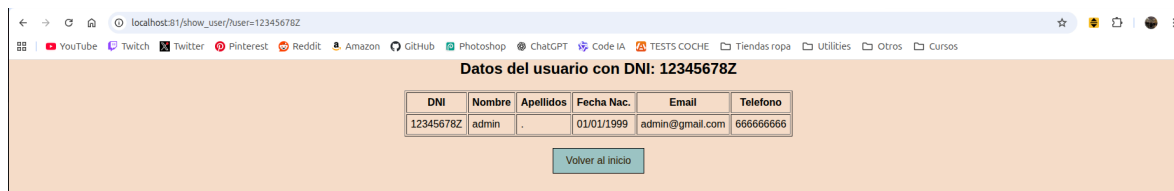


Figura 4. Información sensible del admin revelada.

2.2. CAMINO 2 (MANIPULACIÓN DE PARÁMETROS EN URL)

Además, como segundo camino alternativo al mismo resultado, navegando por la aplicación detectamos que la página de `show_user` acepta un parámetro en la URL que corresponde al DNI. Al sustituir dicho parámetro por el DNI que ya conocemos del primer paso, se mostró una tabla con los datos personales asociados (nombre, apellidos, fecha de nacimiento, email, teléfono) del usuario `admin` (Figura 5).

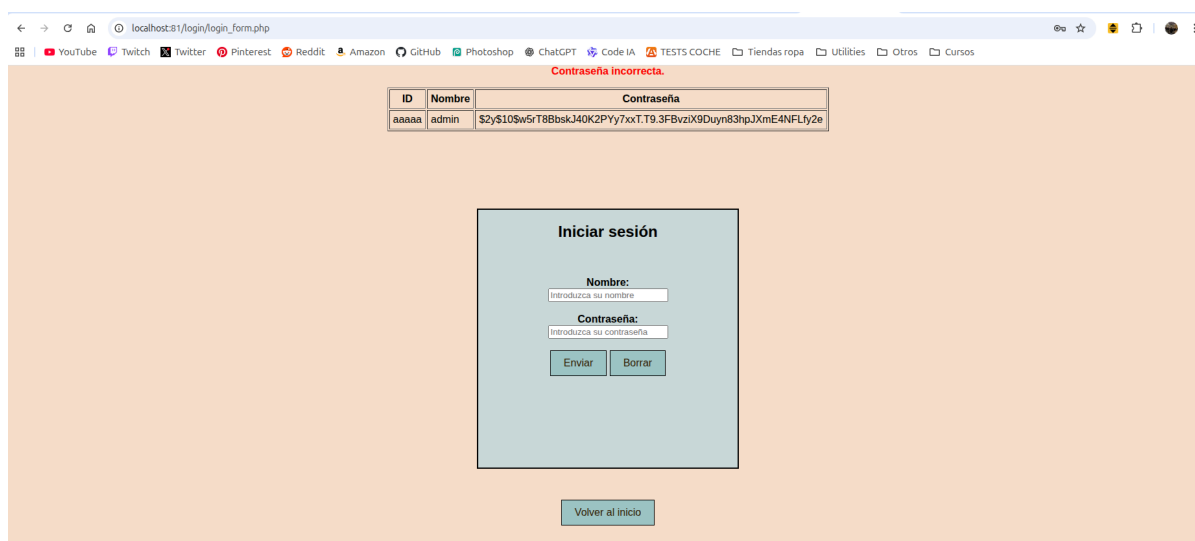


The screenshot shows a web browser at the URL `localhost:81/show_user/user=12345678Z`. The page title is "Datos del usuario con DNI: 12345678Z". Below the title is a table with user details. At the bottom of the table is a button labeled "Volver al inicio".

DNI	Nombre	Apellidos	Fecha Nac.	Email	Telefono
12345678Z	admin	.	01/01/1999	admin@gmail.com	666666666

Figura 5. Información sensible del admin por medio de manipulación del parámetro en URL.

La aplicación parece construir la consulta SQL de autenticación utilizando directamente el nombre de usuario introducido, algo equivalente a `SELECT * FROM usuarios WHERE nombre = <valor_parámetro_user>`. Aprovechamos este comportamiento para introducir el nombre del `admin`, de modo que la consulta devolviese la tabla basada en dicho usuario. El problema es que la aplicación envía todos los campos resultantes de la consulta al frontend, en lugar de limitarse a comprobar internamente si la contraseña es correcta. Como consecuencia, la interfaz muestra información sensible del usuario, incluida la contraseña hashada. (Figura 6).



The screenshot shows a web browser at the URL `localhost:81/login_form.php`. The page has a red error message "Contraseña incorrecta." at the top. Below it is a table with user details. At the bottom of the table is a button labeled "Volver al inicio".

ID	Nombre	Contraseña
aaaaa	admin	\$2y\$10\$w5rT8Bbsk340K2Py7xxT.T9.3FBvziX9Duy83hpJXmE4NFLfy2e

Figura 6. Información sensible del admin revelada.

Para poder llevar a cabo el ataque de fuerza bruta y tratar de recuperar las contraseñas originales a partir de los hashes obtenidos, fue necesario disponer de un diccionario adecuado. Para ello descargamos el paquete SecLists, que incluye colecciones de contraseñas reales filtradas en diversas brechas de datos. Dentro de este paquete se encuentra el diccionario rockyou.txt, uno de los más utilizados en auditorías de seguridad. Tras instalar SecLists mediante snap, copiamos y descomprimos el archivo correspondiente para obtener el fichero rockyou.txt, que utilizamos como base del ataque.

Una vez obtenidos los hashes de las contraseñas de los usuarios, se han guardado en un archivo llamado “hashes.txt”. Estos hashes comienzan por \$2y\$10\$, lo que indica que fueron generados con bcrypt. Se ha utilizado el siguiente comando:

```
hashcat -m 3200 -a 0 hashes.txt rockyou.txt
```

Para intentar averiguar las contraseñas originales empleamos Hashcat, configurado en el modo 3200 (correspondiente a bcrypt) junto con un ataque por diccionario usando rockyou.txt. Como bcrypt es un algoritmo especialmente lento, el proceso completo tarda varios días.

Para comprobar que todo funcionaba correctamente, de forma rápida, añadimos manualmente la palabra “test”, que es la contraseña original de admin, al principio del diccionario, utilizando el comando:

```
sed -i '1s/^/test\n/' rockyou.txt
```

Hashcat logró emparejarla con uno de los hashes, el del admin (Figura 7), demostrando que la técnica era válida siempre que el diccionario contenga la contraseña correcta. Aunque, más lento, se pueden realizar ataques de fuerza bruta o mixtos, modificando el valor -a.

```
lker@IKER:~$ hashcat -m 3200 -a 0 hashes.txt rockyou.txt --potfile-disable
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 5.0+debian Linux, None+Asserts, RELOC, SPIR, LLVM 16.0.6, SLEEP, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
=====
* Device #1: cpu-haswell-13th Gen Intel(R) Core(TM) i7-13620H, 6801/13666 MB (2048 MB allocatable), 16MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 72

Hashes: 1 digests; 1 unique digests; 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Single-Hash
* Single-Salt

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 0 MB

Dictionary cache hit:
* Filename..: rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921502
* Keyspace...: 14344385

$2y$10$w5rT8BbskJ40K2Pyy7xxT.T9.3FBvziX9Duyn83hpJXmE4NFLfy2e:test

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 3200 (bcrypt $2*$, Blowfish (Unix))
Hash.Target.....: $2y$10$w5rT8BbskJ40K2Pyy7xxT.T9.3FBvziX9Duyn83hpJXmE4NFLfy2e
Time.Started.....: Fri Nov 14 00:40:28 2025 (1 sec)
Time.Estimated...: Fri Nov 14 00:40:29 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 246 H/s (3.84ms) @ Accel:16 Loops:4 Thr:1 Vec:1
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 256/14344385 (0.00%)
Rejected.....: 0/256 (0.00%)
Restore.Point...: 0/14344385 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:1020-1024
Candidate.Engine.: Device Generator
Candidates.#1...: test -> samsung
Hardware.Mon.#1..: Temp: 63c Util: 87%
```

Figura 7. Resultados del ataque por diccionario.

3. CONCLUSIÓN DEL ATAQUE

En resumen, las vulnerabilidades raíz que han permitido atacar es la falta de validación y saneamiento de entradas en el parámetro nombre y en el parámetro dni de la URL, así como, la mala gestión de las consultas SQL y la exposición de los resultados al frontend.

Finalmente, las vulnerabilidades nos permiten obtener los hashes de las contraseñas almacenadas en la base de datos. Aunque bcrypt ofrece una protección fuerte, disponer de los hashes permite intentar romperlos con herramientas, lentas, pero efectivas, como Hashcat, especialmente si los usuarios utilizan contraseñas débiles o habituales. Esto representa un riesgo grave, ya que comprometer estos hashes puede abrir la puerta al acceso no autorizado a todas las cuentas de la aplicación.